

Kafka Hands-On Lab Guide

Prerequisites

- Docker and Docker Compose installed
- Go 1.24.0 installed
- Terminal/Command line access
- Text editor or IDE

Lab Structure

```
kafka-lab/
├─ docker-compose-single.yml      # Single broker setup
├─ docker-compose-cluster.yml     # 3-broker cluster
├─ go.mod                        # Go module file
├─ go.sum                        # Go dependencies
├─ examples/                     # Go code examples
│   └─ producer.go
│   └─ consumer.go
│   └─ patterns.go
├─ scripts/                       # Helper scripts
│   └─ setup.sh
│   └─ create-topics.sh
│   └─ test-producer.sh
│   └─ test-consumer.sh
└─ README.md                      # This file
```

Part 1: Setup and Basic Operations

Step 1: Start Single Broker Kafka

```
# Create project directory
mkdir kafka-lab && cd kafka-lab

# Start single broker
docker-compose -f docker-compose-single.yml up -d

# Wait for services to be ready (30 seconds)
sleep 30

# Check if containers are running
docker ps

# Expected output:
# - zookeeper (port 2181)
# - kafka (port 9092)
# - kafka-ui (port 8080)
```

Step 2: Access Kafka UI

Open browser: <http://localhost:8080>

You should see:

- Broker list
- Topics (empty initially)
- Consumer groups

Step 3: Create Your First Topic

```
# Enter kafka container
docker exec -it kafka bash

# Inside container - create topic
kafka-topics --bootstrap-server localhost:9092 \
  --create \
  --topic my-first-topic \
  --partitions 3 \
  --replication-factor 1

# Describe the topic
kafka-topics --bootstrap-server localhost:9092 \
  --describe \
  --topic my-first-topic

# Exit container
exit
```

Expected Output:

```
Topic: my-first-topic
PartitionCount: 3
ReplicationFactor: 1
Partition: 0    Leader: 1    Replicas: 1    Isr: 1
Partition: 1    Leader: 1    Replicas: 1    Isr: 1
Partition: 2    Leader: 1    Replicas: 1    Isr: 1
```

Step 4: Produce Messages via CLI

```
# Start console producer
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic

# Type messages (press Enter after each):
Hello Kafka!
This is message 2
Testing message 3

# Press Ctrl+C to exit
```

Step 5: Consume Messages via CLI

```
# Terminal 1: Start consumer from beginning
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic \
  --from-beginning

# You should see all 3 messages

# Terminal 2: Start another producer
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic

# Type: New message 4

# Check Terminal 1 - it should show the new message
```

Step 6: Consumer Groups

```
# Terminal 1: Consumer with group
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic \
  --group my-consumer-group \
  --from-beginning

# Terminal 2: Another consumer in same group
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic \
  --group my-consumer-group

# Terminal 3: Produce messages
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic my-first-topic

# Type several messages and observe:
# - Messages are distributed between consumers
# - Each message goes to only ONE consumer in the group
```

Step 7: Check Consumer Group Status

```
# List consumer groups
docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --list

# Describe specific group
docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group my-consumer-group \
  --describe

# You'll see:
# - Current offset
# - Log end offset
# - Lag (difference)
# - Consumer ID
# - Host
```

Part 2: Go Producer and Consumer

Step 1: Initialize Go Module

```
# In your project directory
mkdir examples && cd examples

# Initialize Go module
go mod init kafka-lab

# Install kafka-go
go get github.com/segmentio/kafka-go@v0.4.48
```

Step 2: Simple Producer

Create `examples/simple-producer.go`:

```

package main

import (
    "context"
    "fmt"
    "log"
    "time"

    "github.com/segmentio/kafka-go"
)

func main() {
    writer := &kafka.Writer{
        Addr:      kafka.TCP("localhost:9092"),
        Topic:     "go-topic",
        Balancer: &kafka.Hash{},
    }
    defer writer.Close()

    ctx := context.Background()

    // Send 10 messages
    for i := 0; i < 10; i++ {
        msg := kafka.Message{
            Key:    []byte(fmt.Sprintf("key-%d", i)),
            Value: []byte(fmt.Sprintf("Message number %d from Go", i)),
            Time:   time.Now(),
        }

        err := writer.WriteMessages(ctx, msg)
        if err != nil {
            log.Fatal("failed to write message:", err)
        }

        fmt.Printf("Sent: key=%s, value=%s\n", msg.Key, msg.Value)
        time.Sleep(500 * time.Millisecond)
    }

    fmt.Println("All messages sent!")
}

```

Run it:

```
# Create topic first
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic go-topic --partitions 3 --replication-factor 1

# Run producer
go run simple-producer.go
```

Step 3: Simple Consumer

Create `examples/simple-consumer.go`:


```

package main

import (
    "context"
    "fmt"
    "log"

    "github.com/segmentio/kafka-go"
)

func main() {
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers:  []string{"localhost:9092"},
        Topic:     "go-topic",
        GroupID:   "go-consumer-group",
        MinBytes:  10e3, // 10KB
        MaxBytes:  10e6, // 10MB
    })
    defer reader.Close()

    ctx := context.Background()

    fmt.Println("Consumer started, waiting for messages...")

    // Read 10 messages
    for i := 0; i < 10; i++ {
        msg, err := reader.ReadMessage(ctx)
        if err != nil {
            log.Fatal("failed to read message:", err)
        }

        fmt.Printf("Received: partition=%d, offset=%d, key=%s, value=%s\n",
            msg.Partition, msg.Offset, string(msg.Key), string(msg.Value))
    }

    fmt.Println("Consumed 10 messages successfully!")
}

```

Run it:

```
# Terminal 1: Run consumer
go run simple-consumer.go

# Terminal 2: Run producer
go run simple-producer.go
```

Part 3: Advanced Patterns

Lab 3.1: Producer with Acknowledgment Levels

Create `examples/acks-test.go`:

```
package main

import (
    "context"
    "fmt"
    "time"

    "github.com/segmentio/kafka-go"
)

func testAcks(acks kafka.RequiredAcks, name string) {
    writer := &kafka.Writer{
        Addr:      kafka.TCP("localhost:9092"),
        Topic:      "acks-test",
        RequiredAcks: acks,
    }
    defer writer.Close()

    ctx := context.Background()
    start := time.Now()

    for i := 0; i < 100; i++ {
        writer.WriteMessages(ctx, kafka.Message{
            Value: []byte(fmt.Sprintf("Message %d", i)),
        })
    }

    duration := time.Since(start)
    fmt.Printf("%s: Sent 100 messages in %v\n", name, duration)
}

func main() {
    // Create topic
    fmt.Println("Testing different ack levels...")

    testAcks(kafka.RequireNone, "acks=0")
    time.Sleep(2 * time.Second)

    testAcks(kafka.RequireOne, "acks=1")
    time.Sleep(2 * time.Second)

    testAcks(kafka.RequireAll, "acks=all")
}
```

Expected Results:

- `acks=0`: Fastest (no wait for acknowledgment)
- `acks=1`: Medium (wait for leader)
- `acks=all`: Slowest (wait for all replicas)

Lab 3.2: Consumer Group Scaling

Create `examples/consumer-group.go`:

```
package main

import (
    "context"
    "fmt"
    "log"
    "os"
    "sync"
    "time"

    "github.com/segmentio/kafka-go"
)

func consumer(id int, wg *sync.WaitGroup) {
    defer wg.Done()

    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers:      []string{"localhost:9092"},
        GroupID:      "scaling-test-group",
        Topic:        "scaling-test",
        CommitInterval: time.Second,
    })
    defer reader.Close()

    ctx := context.Background()

    fmt.Printf("Consumer %d started\n", id)

    for i := 0; i < 20; i++ {
        msg, err := reader.ReadMessage(ctx)
        if err != nil {
            log.Printf("Consumer %d error: %v", id, err)
            break
        }

        fmt.Printf("Consumer %d: partition=%d, offset=%d, value=%s\n",
            id, msg.Partition, msg.Offset, string(msg.Value))

        time.Sleep(100 * time.Millisecond) // Simulate processing
    }

    fmt.Printf("Consumer %d finished\n", id)
}
```

```

func main() {
    numConsumers := 3
    if len(os.Args) > 1 {
        fmt.Sscanf(os.Args[1], "%d", &numConsumers)
    }

    var wg sync.WaitGroup

    fmt.Printf("Starting %d consumers...\n", numConsumers)

    for i := 0; i < numConsumers; i++ {
        wg.Add(1)
        go consumer(i+1, &wg)
    }

    wg.Wait()
    fmt.Println("All consumers finished")
}

```

Test it:

```

# Create topic with 6 partitions
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
    --create --topic scaling-test --partitions 6 --replication-factor 1

# Terminal 1: Start 2 consumers
go run consumer-group.go 2

# Terminal 2: Start 4 consumers
go run consumer-group.go 4

# Terminal 3: Produce messages
docker exec -it kafka kafka-console-producer \
    --bootstrap-server localhost:9092 \
    --topic scaling-test

# Type many messages and observe distribution

```

Lab 3.3: Dead Letter Queue Pattern

Create `examples/dlq-pattern.go`:

```
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "log"
    "math/rand"
    "time"

    "github.com/segmentio/kafka-go"
)

type Order struct {
    OrderID string `json:"orderId"`
    Amount  float64 `json:"amount"`
}

func processOrder(order Order) error {
    // Simulate random failures (20% failure rate)
    if rand.Float64() < 0.2 {
        return fmt.Errorf("processing failed for order %s", order.OrderID)
    }

    fmt.Printf("✓ Successfully processed order %s\n", order.OrderID)
    return nil
}

func main() {
    rand.Seed(time.Now().UnixNano())

    // Main topic reader
    reader := kafka.NewReader(kafka.ReaderConfig{
        Brokers:      []string{"localhost:9092"},
        GroupID:      "dlq-example",
        Topic:        "orders",
        CommitInterval: 0, // Manual commit
    })
    defer reader.Close()

    // DLQ writer
    dlqWriter := &kafka.Writer{
        Addr:  kafka.TCP("localhost:9092"),
```

```

        Topic: "orders-dlq",
    }
    defer dlqWriter.Close()

    ctx := context.Background()

    fmt.Println("DLQ Consumer started...")

    for i := 0; i < 50; i++ {
        msg, err := reader.FetchMessage(ctx)
        if err != nil {
            log.Printf("Error fetching: %v", err)
            break
        }

        var order Order
        json.Unmarshal(msg.Value, &order)

        // Try processing
        err = processOrder(order)
        if err != nil {
            fmt.Printf("✗ Failed: %v - Sending to DLQ\n", err)

            // Send to DLQ
            dlqWriter.WriteMessages(ctx, kafka.Message{
                Key:    msg.Key,
                Value: msg.Value,
                Headers: []kafka.Header{
                    {Key: "error", Value: []byte(err.Error())},
                    {Key: "failed-at", Value: []byte(time.Now().String())},
                    {Key: "original-topic", Value: []byte("orders")},
                },
            })
        }

        // Always commit
        reader.CommitMessages(ctx, msg)
    }

    fmt.Println("Finished processing")
}

```

Test it:


```
# Create topics
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 3 --replication-factor 1

docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic orders-dlq --partitions 1 --replication-factor 1

# Terminal 1: Run DLQ consumer
go run dlq-pattern.go

# Terminal 2: Produce orders
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic orders

# Paste these JSON messages:
{"orderId":"ORD-001","amount":99.99}
{"orderId":"ORD-002","amount":149.50}
{"orderId":"ORD-003","amount":75.25}
# ... (paste 20-30 orders)

# Terminal 3: Check DLQ
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders-dlq \
  --from-beginning \
  --property print.headers=true
```

Part 4: Multi-Broker Cluster

Step 1: Stop Single Broker

```
docker-compose -f docker-compose-single.yml down -v
```

Step 2: Start 3-Broker Cluster

```
docker-compose -f docker-compose-cluster.yml up -d
```

```
# Wait for cluster to be ready
```

```
sleep 30
```

```
# Check brokers
```

```
docker exec -it kafka-1 kafka-broker-api-versions \  
  --bootstrap-server localhost:9092 | grep id
```

Step 3: Create Replicated Topic

```
docker exec -it kafka-1 kafka-topics \  
  --bootstrap-server localhost:9092 \  
  --create \  
  --topic replicated-topic \  
  --partitions 6 \  
  --replication-factor 3
```

```
# Describe to see replica distribution
```

```
docker exec -it kafka-1 kafka-topics \  
  --bootstrap-server localhost:9092 \  
  --describe \  
  --topic replicated-topic
```

```
# Expected output shows Leader and Replicas across brokers 1,2,3
```

Step 4: Test Fault Tolerance

```
# Terminal 1: Start consumer
docker exec -it kafka-1 kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic replicated-topic \
  --group fault-test

# Terminal 2: Start producer
docker exec -it kafka-1 kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic replicated-topic

# Send some messages

# Terminal 3: Stop broker 2 (simulate failure)
docker stop kafka-2

# Continue sending/receiving messages - should still work!
# Check topic description to see leadership changes

docker exec -it kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic replicated-topic

# Restart broker
docker start kafka-2

# Check under-replicated partitions
docker exec -it kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --under-replicated-partitions
```

Part 5: Performance Testing

Lab 5.1: Producer Performance

Create `examples/perf-producer.go`:

```
package main

import (
    "context"
    "fmt"
    "time"

    "github.com/segmentio/kafka-go"
)

func main() {
    writer := &kafka.Writer{
        Addr:      kafka.TCP("localhost:9092"),
        Topic:      "perf-test",
        Balancer:   &kafka.Hash{},
        RequiredAcks: kafka.RequireOne,
        Compression: kafka.Lz4,
        BatchSize:  100,
        BatchTimeout: 10 * time.Millisecond,
    }
    defer writer.Close()

    ctx := context.Background()
    numMessages := 10000
    messageSize := 1024 // 1KB

    payload := make([]byte, messageSize)
    for i := range payload {
        payload[i] = 'A'
    }

    fmt.Printf("Sending %d messages of %d bytes each...\n", numMessages, messageSize)

    start := time.Now()

    for i := 0; i < numMessages; i++ {
        msg := kafka.Message{
            Key:    []byte(fmt.Sprintf("key-%d", i)),
            Value:  payload,
        }

        err := writer.WriteMessages(ctx, msg)
        if err != nil {
```

```

        fmt.Printf("Error: %v\n", err)
    }
}

duration := time.Since(start)
throughput := float64(numMessages) / duration.Seconds()
dataSizeMB := float64(numMessages*messageSize) / (1024 * 1024)
dataThroughputMB := dataSizeMB / duration.Seconds()

fmt.Printf("\n=== Results ===\n")
fmt.Printf("Duration: %v\n", duration)
fmt.Printf("Messages/sec: %.2f\n", throughput)
fmt.Printf("Total data: %.2f MB\n", dataSizeMB)
fmt.Printf("Throughput: %.2f MB/sec\n", dataThroughputMB)
}

```

Run it:

```

# Create topic
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
    --create --topic perf-test --partitions 6 --replication-factor 1

# Run test
go run perf-producer.go

```

Lab 5.2: Consumer Performance

```

# Use built-in performance test
docker exec -it kafka kafka-consumer-perf-test \
    --bootstrap-server localhost:9092 \
    --topic perf-test \
    --messages 10000 \
    --threads 4 \
    --group perf-consumer-group

# You'll see:
# - Total messages consumed
# - Messages/sec
# - MB/sec
# - Rebalance time

```

Part 6: Monitoring and Debugging

Lab 6.1: Monitor Consumer Lag

```
# In one terminal, slow consumer
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic perf-test \
  --group slow-consumer \
  --max-messages 1

# In another terminal, fast producer
for i in {1..1000}; do
  echo "message-$i"
done | docker exec -i kafka kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic perf-test

# Check lag
watch -n 1 'docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group slow-consumer \
  --describe'

# You should see LAG increasing
```

Lab 6.2: Reset Consumer Offsets

```
# Stop the consumer (Ctrl+C)

# Reset to beginning
docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group slow-consumer \
  --topic perf-test \
  --reset-offsets \
  --to-earliest \
  --execute

# Verify
docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group slow-consumer \
  --describe

# Start consumer again - it will reprocess from beginning
```

Lab 6.3: Check Broker Health

```
# Check broker logs
docker logs kafka-1 --tail 100

# Check under-replicated partitions
docker exec -it kafka-1 kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --under-replicated-partitions

# Check disk usage
docker exec -it kafka-1 du -sh /var/lib/kafka/data/*

# Get cluster info
docker exec -it kafka-1 kafka-broker-api-versions \
  --bootstrap-server localhost:9092
```

Part 7: Cleanup and Troubleshooting

Cleanup

```
# Stop and remove all containers
docker-compose -f docker-compose-single.yml down -v

# or
docker-compose -f docker-compose-cluster.yml down -v

# Remove all volumes (WARNING: deletes all data)
docker volume prune -f
```

Common Issues and Solutions

Issue 1: "Connection refused" errors

Solution:

```
# Check if containers are running
docker ps

# Check logs
docker logs kafka

# Wait longer for startup (Kafka takes 20-30 seconds)
sleep 30
```

Issue 2: Consumer not receiving messages

Solution:

```
# Check topic exists
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list

# Check messages are in topic
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic YOUR_TOPIC \
  --from-beginning \
  --max-messages 1

# Check consumer group
docker exec -it kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group YOUR_GROUP \
  --describe
```


Issue 3: Out of memory errors

Solution:

```
# Reduce batch sizes in your code
# Increase Docker memory allocation
# Check Docker stats
docker stats
```

Quick Reference Commands

Topic Operations

```
# List topics
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --list

# Create topic
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic TOPIC_NAME --partitions 3 --replication-factor 1

# Delete topic
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --delete --topic TOPIC_NAME

# Describe topic
docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic TOPIC_NAME
```

Producer/Consumer

```
# Console producer
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 --topic TOPIC_NAME

# Console consumer
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic TOPIC_NAME --from-beginning

# Consumer with group
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic TOPIC_NAME --group GROUP_NAME
```

Consumer Groups

```
# List groups
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --list

# Describe group
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 \
  --group GROUP_NAME --describe

# Delete group
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 \
  --group GROUP_NAME --delete
```

Next Steps

1. **Experiment** with different configurations
2. **Modify** the Go examples to fit your use cases
3. **Test** error scenarios (broker failures, network issues)
4. **Monitor** performance metrics
5. **Build** a real application using Kafka

Additional Resources

- Kafka Documentation: <https://kafka.apache.org/documentation/> (<https://kafka.apache.org/documentation/>)
- segmentio/kafka-go: <https://github.com/segmentio/kafka-go> (<https://github.com/segmentio/kafka-go>)
- Kafka UI: <http://localhost:8080> (when running)

Congratulations! You've completed the Kafka hands-on lab. You now have practical experience with:

- Setting up Kafka clusters
- Producing and consuming messages
- Working with consumer groups
- Implementing common patterns
- Performance testing and monitoring